CYBERSECURITY MINI PROJECT

PHISHING URL DETECTION SYSTEM

A beginner-friendly Python project to identify suspicious website links

Project Domain	Cybersecurity / Web Security
Technology	Python 3
Project Type	Mini Project
Prepared by	Name: _____
Department / Class	_____
Institution	_____

Academic project report

TABLE OF CONTENTS

• 1. Abstract

• 2. Introduction

• 3. Problem Statement and Objectives

• 4. Existing and Proposed System

• 5. System Requirements

• 6. Methodology and Working

• 7. Algorithm and Flow

• 8. Python Source Code

• 9. Sample Output and Testing

• 10. Advantages, Limitations and Future Scope

• 11. Conclusion

• 12. References and Viva Questions

1. ABSTRACT

Phishing is a cyberattack in which an attacker creates a fake website or sends a deceptive link to trick people into sharing sensitive information such as passwords, bank details or personal data. This project, called **Phishing URL Detection System**, checks a website address and identifies common warning signs associated with phishing links.

The project is developed using Python. It applies simple rule-based checks, such as checking whether the URL uses HTTPS, contains an IP address, uses an unusually long address, includes suspicious words, or contains an excessive number of special characters. Based on the checks, the program displays a risk score and a warning. It is intended as an educational demonstration and cannot guarantee that a website is safe or malicious.

2. INTRODUCTION

People use websites for banking, shopping, education and communication. Cybercriminals may imitate trusted websites and send links through email, text messages or social media. A user who opens a fake page may be asked to enter confidential information. This technique is known as phishing.

A URL (Uniform Resource Locator) is the address used to access a resource on the internet. Some characteristics of a URL can be warning signs. For example, a link containing an IP address instead of a domain name, many unusual symbols, or words such as 'verify-account' may deserve extra attention. This project demonstrates how basic checks can help users notice suspicious links.

3. PROBLEM STATEMENT AND OBJECTIVES

Problem statement: Users may not easily recognise deceptive website links. A simple tool is needed to demonstrate common URL warning signs before a user visits an unfamiliar website.

Objectives:

- Accept a URL from the user.
- Check for selected phishing indicators.
- Calculate a simple risk score.
- Display a low, medium or high warning level.
- Encourage users to verify links before sharing personal information.

4. EXISTING AND PROPOSED SYSTEM

Existing approach: Users often rely on visual appearance or the message accompanying a link. A fake website may copy the design and logo of a genuine service, making visual identification difficult.

Proposed approach: The proposed Python program analyses the text of a URL using a set of transparent rules. It reports which indicators were found and calculates a risk score. It does not open the URL or collect user credentials.

5. SYSTEM REQUIREMENTS

Hardware: A desktop computer or laptop; 2 GB RAM or above recommended; basic keyboard and display.

Software: Windows, Linux or macOS; Python 3.8 or later; a text editor or IDE such as IDLE or Visual Studio Code. No external Python packages are required.

6. METHODOLOGY AND WORKING

The system follows these steps:

Step	Description
1	The user enters a website URL.
2	The program removes extra spaces and checks that the input has a basic URL format.
3	The URL is converted to lowercase for consistent checking.
4	The program checks selected indicators: HTTPS absence, IP-address host, long URL, suspicious keywords, '@' symbol, and many hyphens.
5	Each indicator adds points to the risk score.
6	The program displays the score, warning level and reasons.

Risk scoring used in this demonstration

Indicator	Points
URL does not use HTTPS	2
Host appears to be an IPv4 address	3
URL length is greater than 75 characters	1
Contains a suspicious keyword	2
Contains '@' symbol	2
Contains three or more hyphens	1

Risk levels: 0–1 = Low warning; 2–4 = Medium warning; 5 or more = High warning. These thresholds are only for this classroom demonstration and are not a verified security rating.

7. ALGORITHM AND FLOW

Algorithm:

1. Start the program.
2. Read the URL from the user.
3. Validate that the input begins with http:// or https://.
4. Extract the host name.
5. Check the URL for each warning indicator.
6. Add the corresponding points to the risk score.
7. Decide the warning level based on the score.
8. Display the result and reasons.
9. Stop.

Flow description: Start → Enter URL → Basic validation → Check indicators → Calculate score → Display warning level and reasons → End.

8. PYTHON SOURCE CODE

Save the following code in a file named **phishing_detector.py**. Run it using Python 3.

```
from urllib.parse import urlparse
import ipaddress

SUSPICIOUS_WORDS = [
    "login", "verify", "account", "update",
    "secure", "banking", "password", "confirm"
]

def check_url(url):
    score = 0
    reasons = []

    parsed = urlparse(url)
    host = parsed.hostname or ""

    # Check whether HTTPS is used
    if parsed.scheme != "https":
        score += 2
        reasons.append("The URL does not use HTTPS.")

    # Check whether the host is an IP address
    try:
        ipaddress.ip_address(host)
        score += 3
        reasons.append("The website uses an IP address as its host.")
    except ValueError:
        pass

    # Check URL length
    if len(url) > 75:
        score += 1
        reasons.append("The URL is unusually long.")

    # Check suspicious words
    lower_url = url.lower()
    if any(word in lower_url for word in SUSPICIOUS_WORDS):
        score += 2
        reasons.append("The URL contains a suspicious keyword.")

    # Check the @ symbol
    if "@" in url:
        score += 2
        reasons.append("The URL contains an @ symbol.")

    # Check excessive hyphens
    if url.count("-") >= 3:
        score += 1
        reasons.append("The URL contains many hyphens.")

    if score <= 1:
        level = "LOW"
    elif score <= 4:
        level = "MEDIUM"
    else:
        level = "HIGH"

    return score, level, reasons


print("PHISHING URL DETECTION SYSTEM")
print("--------------------------------")

url = input("Enter a website URL: ").strip()

if not url.startswith(("http://", "https://")):
    print("Invalid input. Enter a URL beginning with http:// or https://")
else:
    score, level, reasons = check_url(url)

    print("\nRisk score:", score)
    print("Warning level:", level)

    if reasons:
        print("\nWarning signs:")
        for reason in reasons:
```

```
        print("-", reason)
else:
    print("No selected warning signs were detected.")

print("\nNote: This tool cannot guarantee that a website is safe.")
```

9. SAMPLE OUTPUT AND TESTING

Example 1 – URL with fewer warning signs

```
Enter a website URL: https://www.example.com\nRisk score: 0\nWarning level: LOW\nNo selected warning signs were detected.\n\n
```

Example 2 – URL with several warning signs

```
Enter a website URL: http://192.0.2.10/verify-account-login\nRisk score: 9\nWarning level: HIGH\nWarning signs:\n- The URL do
```

Testing checklist

- Test an HTTPS URL with a normal domain.
- Test an HTTP URL.
- Test a URL that contains an IP address.
- Test a long URL.
- Test a URL containing words such as login or verify.
- Test invalid text that does not begin with http:// or https://.

10. ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

Advantages

• Beginner-friendly and easy to understand.
• Uses built-in Python libraries only.
• Explains the warning signs instead of showing only a score.
• Does not visit the submitted website or request personal information.
• Can be extended as a larger cybersecurity project.

Limitations

• Rule-based checks can produce false alarms.
• A phishing URL may not contain any of the selected warning signs.
• HTTPS does not prove that a website is genuine; phishing websites can also use HTTPS.
• The program does not check website reputation, page content, redirects or domain registration details.
• The score is educational and must not be treated as a guarantee of safety.

Future enhancements

• Add a graphical user interface using Tkinter.
• Connect to a trusted threat-intelligence service or URL reputation API, subject to its terms and safe handling of submitted URLs.
• Train and evaluate a machine-learning model using a properly labelled dataset.
• Store test results in a local database.
• Add a browser extension that warns users before opening suspicious links.

11. CONCLUSION

The Phishing URL Detection System demonstrates a basic method for identifying warning signs in website addresses using Python. It accepts a URL, checks selected characteristics, calculates a risk score and displays a warning level. The project introduces important cybersecurity ideas such as phishing awareness, URL analysis and secure browsing. Although it is not a complete phishing detection solution, it provides a useful foundation for learning and future development.

12. REFERENCES

1. Python Software Foundation, Python 3 Documentation – https://docs.python.org/3/
2. Python Standard Library: urllib.parse – https://docs.python.org/3/library/urllib.parse.html
3. Python Standard Library: ipaddress – https://docs.python.org/3/library/ipaddress.html
4. CISA, Avoiding Social Engineering and Phishing Attacks – https://www.cisa.gov/

VIVA QUESTIONS

1. What is phishing?
Phishing is a cyberattack that tricks people into revealing sensitive information through fake messages or websites.

2. What is a URL?
A URL is the address used to locate a resource on the internet.

3. What is HTTPS?
HTTPS is the secure version of HTTP. It encrypts data exchanged between a browser and a website, but it does not prove that the website is trustworthy.

4. Which programming language is used?
Python 3.

5. Does this project guarantee that a URL is safe?
No. It checks only a limited set of warning signs and may miss phishing links.

6. What is a risk score?

It is a simple total of points assigned to warning signs found in the URL.

Important safety note: Use only harmless sample URLs when testing. Do not enter passwords, bank details or other personal information into unknown websites. This project is for educational use.